Day 13 – Backtracking Intro

1️⃣ Backtracking kya hota hai?

👉 Backtracking =
Try karo → agar galat ho gaya → wapas jao (undo karo) → dusra try karo

---

🔥 Real Life Example

👉 Maze me raste dhundhna:

Ek path try kiya

Dead end mila ❌

Wapas aaye

Dusra path try kiya ✅

👉 Ye hi backtracking hai

2️⃣ Recursion vs Backtracking

Feature	Recursion	Backtracking
Concept	Function calls	Try + Undo
Use	Simple problems	Complex problems
Example	Factorial	N-Queens

3️⃣ Core Idea

Choose → Explore → Unchoose (Backtrack)

4️⃣ Basic Example – Print All Subsets

(Already Day 12 me dekha, but yahi backtracking ka base hai)

```java
package Day_13;

public class Subsets {

    public static void printSubsets(String str, int i, String curr){

        if(i == str.length()){
            System.out.println(curr);
            return;
        }

        // choose
        printSubsets(str, i+1, curr + str.charAt(i));

        // unchoose (backtrack)
        printSubsets(str, i+1, curr);
    }

    public static void main(String[] args) {
        printSubsets("ab", 0, "");
    }
}
```

5️⃣ Example – Find Permutations (Important 🔥)

👉 Permutation = order matter karta hai

Example:

"abc" → abc, acb, bac, bca, cab, cba

Java Code – Permutations

```java
public class Permutations {

    public static void permute(String str, String ans){

        if(str.length() == 0){
            System.out.println(ans);
            return;
        }

        for(int i = 0; i < str.length(); i++){

            char curr = str.charAt(i);

            String newStr = str.substring(0, i) + str.substring(i+1);

            permute(newStr, ans + curr);
        }
    }

    public static void main(String[] args) {
        permute("abc", "");
    }
}
```

🔥 Concept Samjho

Har step pe ek character choose karte ho

Baaki string me recursion call karte ho

Har path ek permutation deta hai

6️⃣ Key Backtracking Template

```
void function(...){

   if(base case){
      print / return;
   }

   for(choices){

      // choose
      make change

      // explore
      function(...)

      // unchoose (backtrack)
      undo change
   }
}
```

7️⃣ Important Points

✅ Backtracking = recursion + undo
✅ Har step me multiple choices
✅ Tree structure banta hai

Problem 1 – Print All Permutations of String

💡 **Concept**

👉 Har step pe ek character fix karo
👉 Baaki string ke permutations nikalo

✅ **Code**

```
1. public class StringPermutation {

    public static void permute(String str, String ans){

        if(str.length() == 0){
            System.out.println(ans);
            return;
        }

        for(int i = 0; i < str.length(); i++){

            char curr = str.charAt(i);

            String newStr = str.substring(0, i) + str.substring(i+1);

            permute(newStr, ans + curr);
        }
    }

    public static void main(String[] args) {
        permute("abc", "");
    }
}
```

Problem 3 – Permutations of Array

💡 **Concept**

👉 Swap technique use karte hain (Backtracking)

```java
package Day_13;

public class ArrayPermutation {

    public static void permute(int[] arr, int index){

        if(index == arr.length){
            // print array
            for(int num : arr){
                System.out.print(num + " ");
            }
            System.out.println();
            return;
        }

        for(int i = index; i < arr.length; i++){

            // swap
            int temp = arr[index];
            arr[index] = arr[i];
            arr[i] = temp;

            permute(arr, index + 1);

            // backtrack (undo swap)
            temp = arr[index];
            arr[index] = arr[i];
            arr[i] = temp;
        }
    }

    public static void main(String[] args) {

        int[] arr = {1,2,3};
        permute(arr, 0);
    }
}
```

Problem 4 – Count Total Permutations

💡 **Concept**

👉 Total permutations = **n! (factorial)**

✅ Approach 1 – Using Recursion

```
1. public class CountPermutation {

    public static int count(int n){

        if(n == 0) return 1;

        return n * count(n-1);
    }

    public static void main(String[] args) {
        System.out.println(count(3)); // 6
    }
}
```

✅ Approach 2 – Count via Backtracking

```
public class CountPermutationBT {

    static int count = 0;

    public static void permute(String str, String ans){

        if(str.length() == 0){
            count++;
            return;
        }

        for(int i = 0; i < str.length(); i++){

            char curr = str.charAt(i);
            String newStr = str.substring(0, i) + str.substring(i+1);

            permute(newStr, ans + curr);
        }
    }

    public static void main(String[] args) {
        permute("abc", "");
        System.out.println(count);
    }
}
```

🎯 Day 13 Summary

Problem	Concept
Subsets	include / exclude
String Permutation	pick one char
Array Permutation	swap + backtrack
Count Permutations	n!